LM-Backend Presentation

Adaptive Dirichlet–Multinomial Anomaly Detection for IoT Traffic

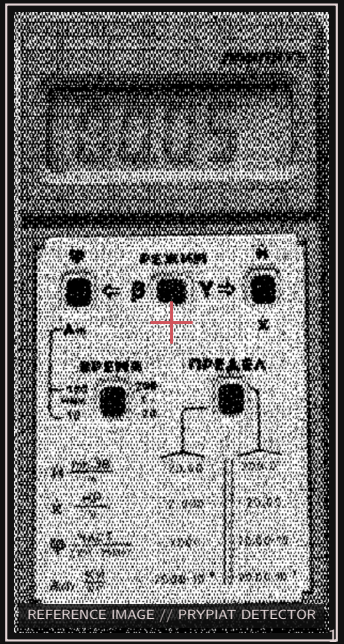
`[ COUNT / MODEL / DETECT / ADAPT ]`

Working system, experimental evidence,
and the route to a defensible final evaluation

Fatemeh Mahvari

Supervisor: Professor Mauro Migliardi

// **28 September 2026**



REFERENCE IMAGE // PRYPIAT DETECTOR

Presentation map

0 // Project context

Goal, pipeline, current development evidence, and the role of LM.

I // Fitted ψ

The dispersion parameter, fitted values, and the baseline paper's threshold.

II // Row-wise backend

Why windows remain separate and how the project adapted the scalar LM code.

III // Evidence

Timing boundaries, runtime results, optimization, and numerical agreement.

Conclusion

What the measurements support—and what they do not.

Project context: goal, pipeline, and current result

LM-IDNet learns one device's normal traffic without attack examples during training, then flags later windows that are unlikely under that model.



- Counts: TCP, ordinary UDP, SSDP, and ARP.
- Model: Dirichlet–multinomial normal behaviour.
- Decision: separately calibrated lower-tail likelihood threshold.
- Labels: used only after scoring for evaluation.

Labelled development F1

	Static	Periodic refit
Chromecast	61.22%	80.70%
Samsung	78.69%	77.31%

Where the LM backend fits



LM does

Evaluate close log-Gamma differences inside the Dirichlet–multinomial likelihood, with explicit numerical error control.

LM does not

Choose the traffic representation, update alpha, derive ψ, define the paper's cutoff, set the anomaly threshold, or make an unchanged detector statistically better.

The backend receives one row per window

$$\mathbf{x}_r = \left(x_{r,\mathrm{TCP}}, x_{r,\mathrm{UDP}}, x_{r,\mathrm{SSDP}}, x_{r,\mathrm{ARP}}\right)$$

- SSDP is separated from ordinary UDP.
- Silent observed windows are zero rows.
- Missing capture coverage is excluded.

Dataset	Days	Windows	Silent	Attack-labelled
D-Link camera	12	1,692	0	–
Chromecast	17	2,349	187	38
Samsung camera	12	1,645	49	82

Since D-Link is unlabelled, it is used only to benchmark the LM algorithm, not in the end-to-end detector pipeline.

Dataset sources

UNSW Chromecast and Samsung camera: https://iotanalytics.unsw.edu.au/attack-data.html

D-Link Camera 5: https://data.mendeley.com/datasets/84cc8grtkt/1

The dispersion parameter and the numerical regime

Dirichlet–multinomial model

The protocol-probability vector may vary between windows. This permits more row-to-row variation than a fixed-probability Multinomial model.

$$A = \sum_k \alpha_k, \qquad \psi = \frac{1}{A}$$

The project reports ψ directly as the fitted Dirichlet–multinomial dispersion parameter throughout the comparison.

LM numerical regime

LM accepts positive $\boldsymbol{\alpha}$ and the corresponding ψ, then evaluates the same likelihood as the reference backend. The paper's cutoff is not an LM precondition.

Why small ψ matters

Smaller ψ means larger concentration and a greater chance that subtracting large, nearby log-Gamma values will lose precision.

Baseline paper and its empirical rule

Main experimental baseline

S. Al-Haj Baddar, A. Languasco and M. Migliardi, ICNC 2025, pp. 28–32, doi: 10.1109/ICNC64010.2025.10993688.

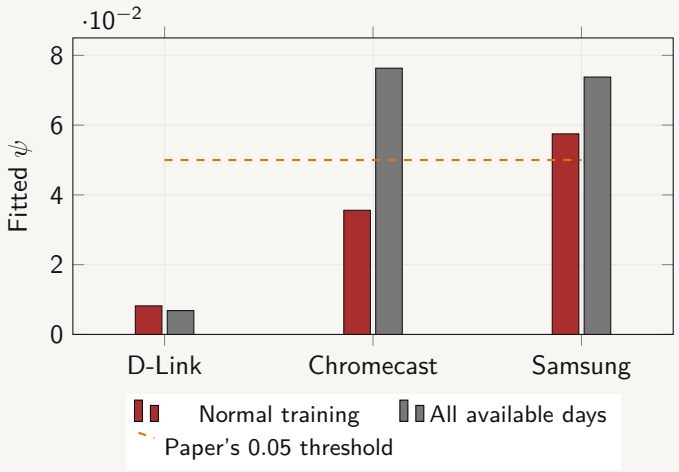
S. Al-Haj Baddar, A. Languasco, and M. Migliardi

- D-Link Camera 5 traffic
- Ten-minute, four-protocol vectors
- Dirichlet–multinomial fitting
- LM likelihood comparison

Paper-defined threshold: $\psi < 0.05$

The paper defines a dataset as overdispersed when its fitted ψ is below 0.05. Numerical failure is not triggered at that exact line: its likelihood increases progressively as ψ becomes smaller.

Aggregate fitted ψ: training versus all days



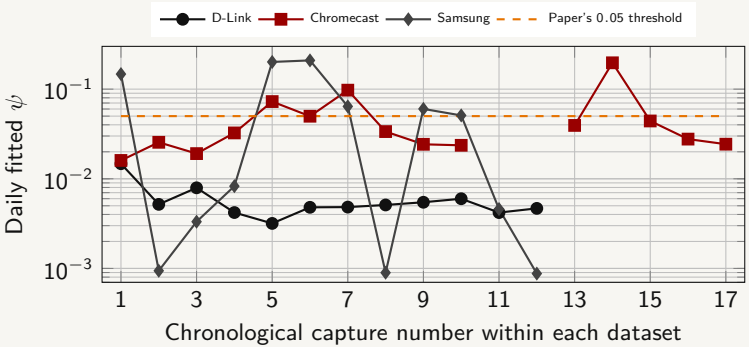
Below 0.05

D-Link in both scopes and Chromecast in normal training satisfy the paper's condition.

Read the scopes

All-days fits mix partitions and include attack periods for the labelled devices. D-Link has the smallest aggregate ψ and the greatest numerical risk among these fits.

Day-to-day change is change in fitted ψ



What is reported

Each point is the day's fitted Dirichlet–multinomial dispersion parameter.

Numerical meaning

The smaller the daily ψ, the greater the chance that conventional log-Gamma likelihood evaluation loses precision. It is not a detector output.

Why the windows must remain separate

Two observed windows

$$\mathbf{x}_1 = (2, 0), \qquad \mathbf{x}_2 = (1, 1)$$

Window 1 has no B; window 2 contains both A and B.

After pooling

$$\mathbf{x}_1 + \mathbf{x}_2 = (3, 1)$$

The total mixture remains, but the change between windows disappears.

The detector's scientific question

What protocol mixtures occur in individual windows, how much do those windows vary, and is a later window plausible under the same device model?

- Each row is one observation for fitting.
- Each later row is one anomaly decision.
- Pooling is suitable only for a different question: long-period total composition.

Evaluate rows, then add likelihoods

For separate observations, probabilities multiply and log-likelihoods add:

$$\ell_{\text{rows}}(\boldsymbol{\alpha}) = \ell(\mathbf{x}_1; \boldsymbol{\alpha}) + \ell(\mathbf{x}_2; \boldsymbol{\alpha}).$$

Row-wise objective

$$K((2,0);\boldsymbol{\alpha}) + K((1,1);\boldsymbol{\alpha})$$

$$-2.89037$$

Pooled objective

$$K((3,1);\boldsymbol{\alpha})$$

$$-2.99573$$

Here $\boldsymbol{\alpha} = (1,1)$. Neither number is inherently "more accurate": they are different likelihoods for different statistical questions. The project needs the row-wise one.

Original capsule versus project objective

	Original LM-wrapped capsule	Production project
Minka update	Uses the original rows	Uses the original rows
Convergence likelihood	`find_X` sums columns, then one scalar LM call evaluates the pooled vector	Matrix interface evaluates every row, then sums the returned likelihood values
Calls per iteration	Old and candidate pooled likelihoods	One new row-wise matrix likelihood; previous value is retained
Meaning	One long-period composition vector	Separate time-window observations under one shared model

The distinction

The original estimator does use rows in its update. The difference is the likelihood objective used to decide convergence, not whether rows exist at all.

Project integration: preserve rows, reuse setup



Earlier adapter

For every row: prepare LM settings, validate, evaluate, release. Correct, but alpha- and precision-dependent setup was repeated.

Current adapter

Prepare shared settings once, validate and evaluate each row separately, sum the values, then release the workspace once.

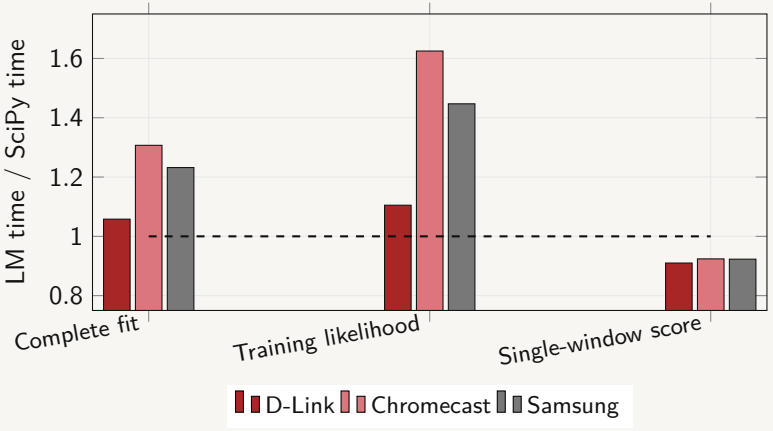
Zero counts, file loading, allocation failures, dimensions, finite results, memory ownership, and reproducible compilation remain checked.

Four timers that must not be confused

Measurement	Included	Excluded
Complete model fit	Alpha initialization, initial likelihood, Minka updates, iterative row-wise likelihoods, convergence checks	PCAP preprocessing, JSON loading, backend construction, saving, calibration, scoring
Training likelihood	One fixed-alpha call on the complete training matrix	Initialization, updates, iterations, coefficient, file I/O
Single-window score	Full log-probability for one non-silent row	Window loading, threshold comparison, event output, evaluation
Original printed time	Two pooled likelihood calls per Minka iteration	Minka update, setup outside calls, data loading, total fit wall time

The reported "complete model fit" is not the complete command-line pipeline; packet preprocessing is excluded.

Production runtime is workload-dependent



Matrix work

LM is slower for the three large row-wise likelihoods and complete fits.

Online work

LM is about 8–9% faster for one complete single-window score.

Row-wise adapter optimization

Before and after

Operation	Earlier LM	Current LM
846-row likelihood	114.68 μs	55.58 μs
Complete fit	0.30410 s	0.19486 s

51.5% and **35.9%** reductions, with identical row-wise results across 1,000 stress matrices.

Agreement now; protection near the boundary

D-Link production agreement

0 / 270
decision mismatches

Relative alpha	3.86×10^{-7}
Absolute score	1.09×10^{-5}
Threshold	2.89×10^{-6}

Absolute error against high precision

ψ	SciPy error	LM error
8.18×10^{-3}	3.07×10^{-10}	3.91×10^{-10}
10^{-8}	2.50×10^{-7}	1.50×10^{-8}
10^{-10}	2.83×10^{-5}	5.65×10^{-8}
10^{-12}	2.02×10^{-3}	5.58×10^{-5}
10^{-14}	3.37×10^{-1}	1.03×10^{-3}

On current data, both backends are accurate. LM's numerical advantage becomes material at extremely small ψ, where cancellation grows.

Conclusion: the defensible LM-backend claim

1. ψ **is the reported dispersion parameter.** The paper's 0.05 threshold is shown directly on the aggregate and daily values.
2. **Rows are part of the detector's meaning.** Evaluating each window and then adding likelihoods preserves the variation that pooling removes.
3. **The integration is validated, not universally faster.** Shared setup materially improved the row-wise adapter; LM is faster for the measured single-window score, while SciPy is faster for current matrix fits.
4. **Numerical value depends on regime.** Current decisions agree, and LM provides stronger protection as ψ becomes smaller and conventional cancellation risk increases.

One-sentence result

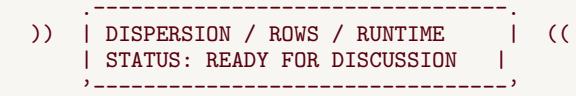
The optimized native backend preserves the project's row-wise objective and numerical decisions; its speed advantage is conditional on the operation and calling architecture.

Claim boundary

No end-to-end pipeline or edge-device speed claim has been made.

Questions and discussion

Questions?



Backup: exact production runtimes

Operation	Dataset	SciPy	LM	LM/SciPy	Difference
Complete fit	D-Link	180.59 ms	191.04 ms	1.058	+5.8%
	Chromecast	194.40 ms	254.17 ms	1.307	+30.7%
	Samsung	34.47 ms	42.46 ms	1.232	+23.2%
Matrix likelihood	D-Link	49.79 μs	55.02 μs	1.105	+10.5%
	Chromecast	52.12 μs	84.69 μs	1.625	+62.5%
	Samsung	31.57 μs	45.69 μs	1.447	+44.7%
Single score	D-Link	13.18 μs	12.00 μs	0.910	−9.0%
	Chromecast	13.22 μs	12.22 μs	0.924	−7.6%
	Samsung	13.20 μs	12.18 μs	0.923	−7.7%

Five complete fits per backend; warmed isolated calls measured in five batches. Backend construction and one-time coefficient loading occurred before timing.

Backup: fit workloads

Dataset	Matrix	Iterations	Initial likelihood	Final likelihood
D-Link	846×4	1,882	−477,950.796	−476,052.589
Chromecast	867×4	1,950	−1,464,261.145	−1,462,542.173
Samsung	432×4	580	−191,965.774	−191,562.039

Interpretation

Absolute fit times depend on both matrix size and iteration count. Likelihood magnitudes also depend on the data and row count; they are convergence values, not a ranking of device models.

Backup: native integration changes

Row-wise production path

- Matrix entry point over independent rows
- Shared alpha/precision setup once per matrix
- Same scalar LM numerical engine per row
- Previous convergence likelihood retained

Safety and operation

- Exact zero-count identity
- Checked dimensions, tables, and allocations
- Native errors returned to Python
- Explicit workspace release
- Reproducible shared-library rebuild

Preserved reference

The professor's files under `lm-algorithm/code/` remain unchanged; the application-facing implementation lives in the project's native runtime copy.